

Les arbres: structure et algorithmes

rouanehm@gmail.com

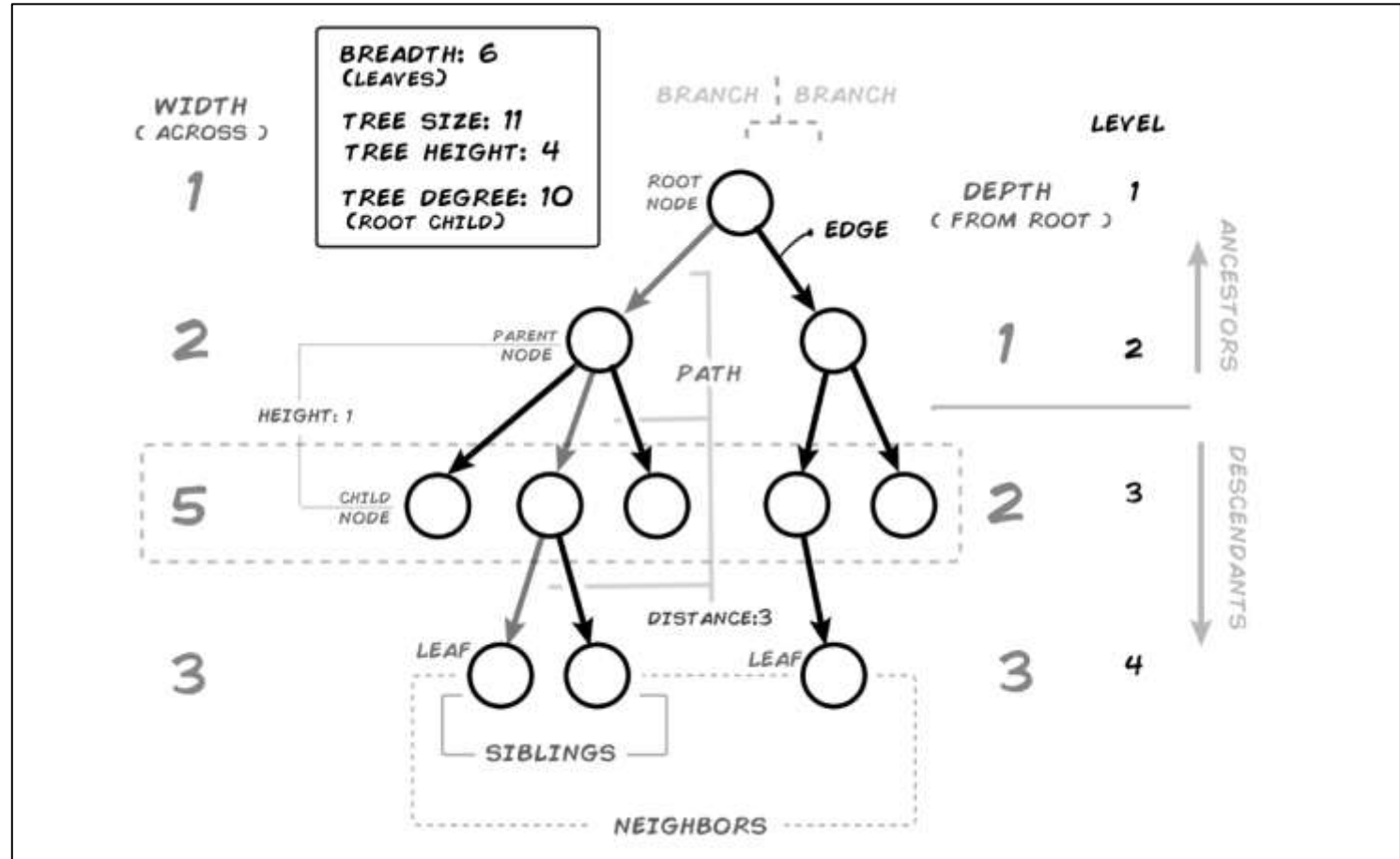
Les arbres

- Des nœuds reliés par des arrêtes
- Les types de nœuds dans un arbre:
 - Des nœuds internes avec des fils
 - Des nœuds feuilles sans fils
 - Un nœud racine, le seul nœud qui n'a pas de père.



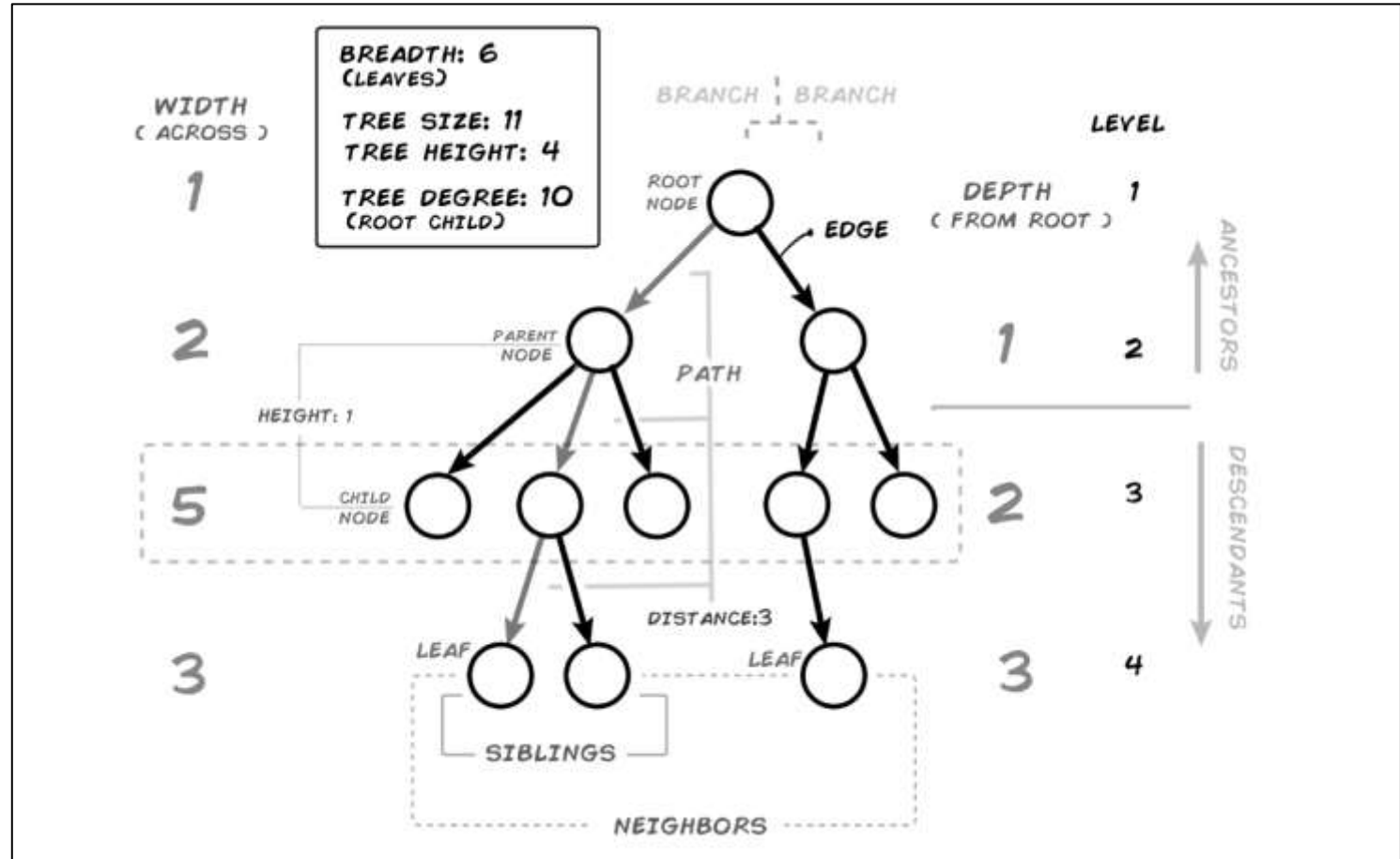
Quelques définitions associées à l'arbre

- Profondeur d'un nœud : la distance en terme de nombre d'arrêtes entre le nœud racine et le nœud concerné.
- Hauteur : La plus grande profondeur d'une feuille de l'arbre
- Taille d'un arbre : le nombre de nœuds (tous types confondus).



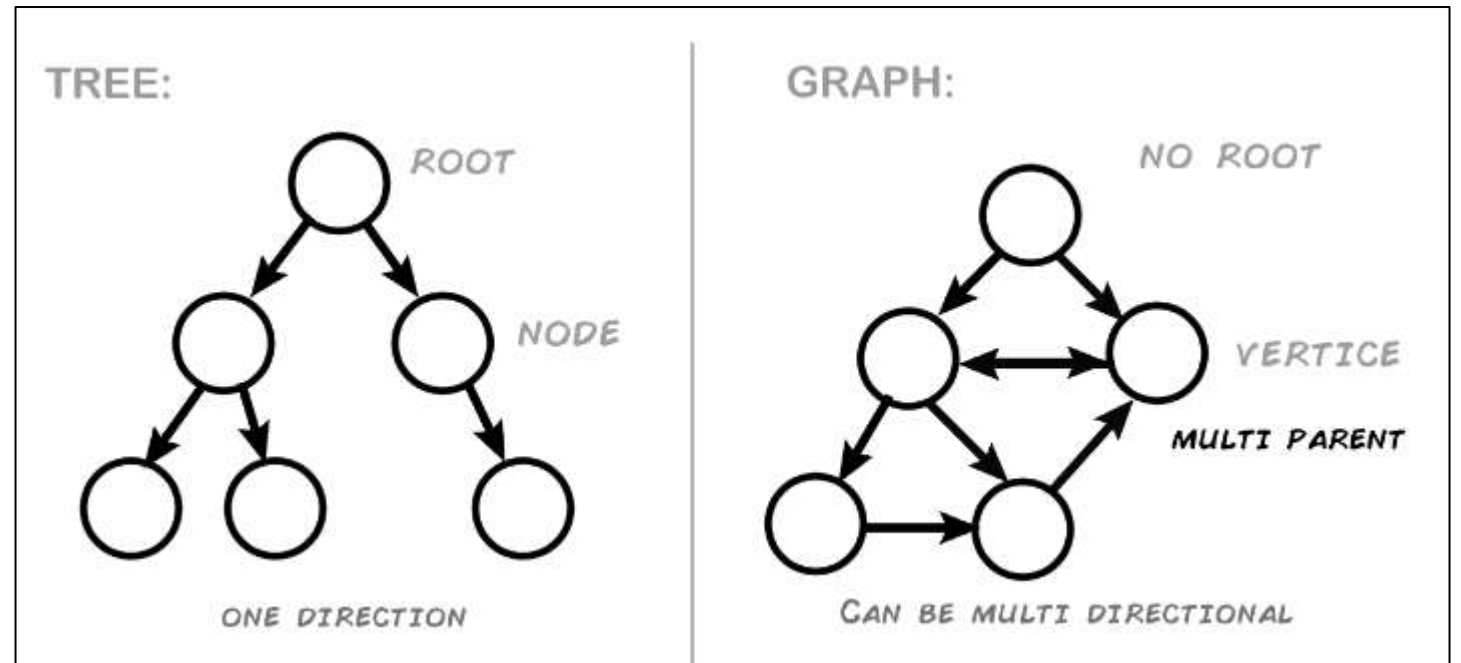
Quelques définitions associées à l'arbre

- Étiquette : certains arbres associent des labels aux arrêtes. Ces labels peuvent être simples (nombre: poids) ou des structures complexes (objets)
- Niveau : un ensemble de nœud de même profondeur
- Descendants : tous les nœuds des sous arbres.



Arbre versus un graph

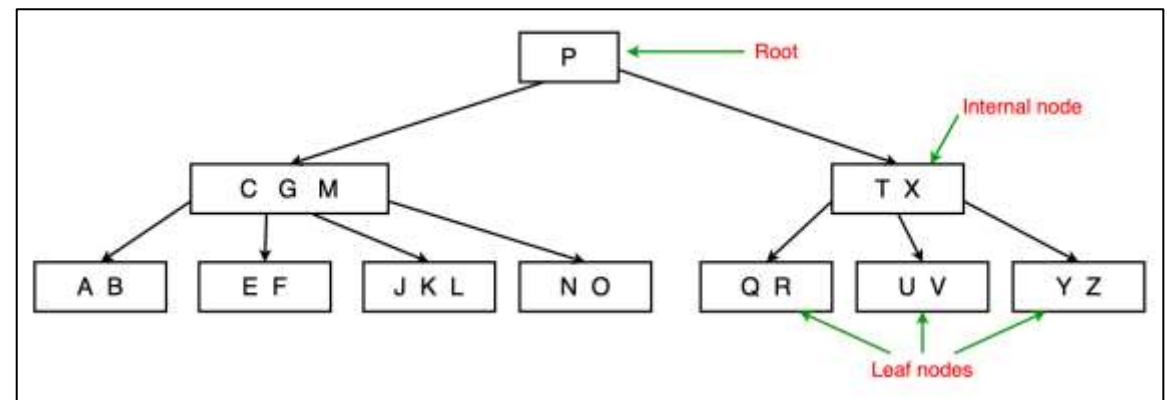
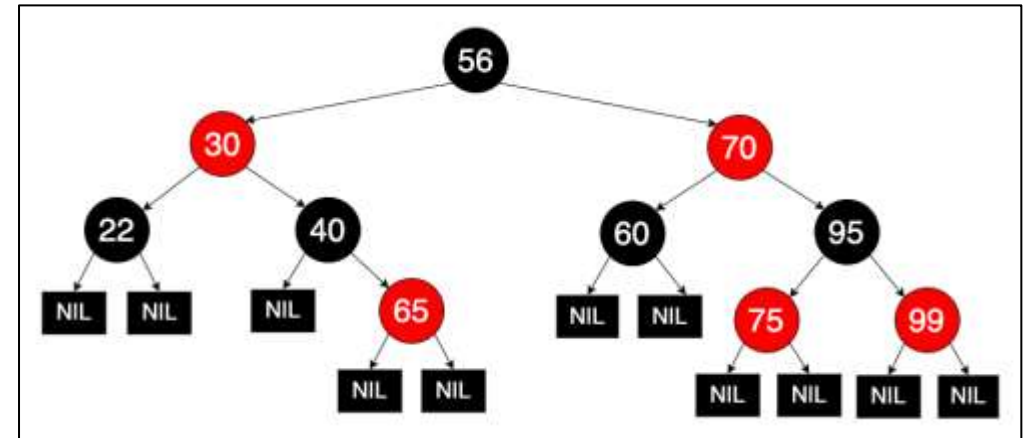
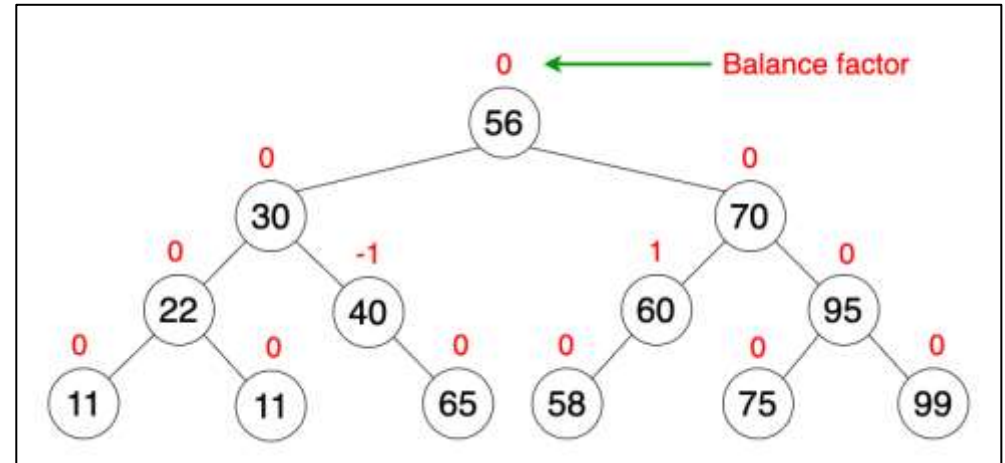
- Dans un graphe il n'y a pas de racine comme dans un arbre.
- Un arbre est un graphe sans cycle muni d'un seul point d'entrée: la racine.
- Dans un graphe, souvent les connections (arrêtes) sont bidirectionnelles.



Types d'arbres

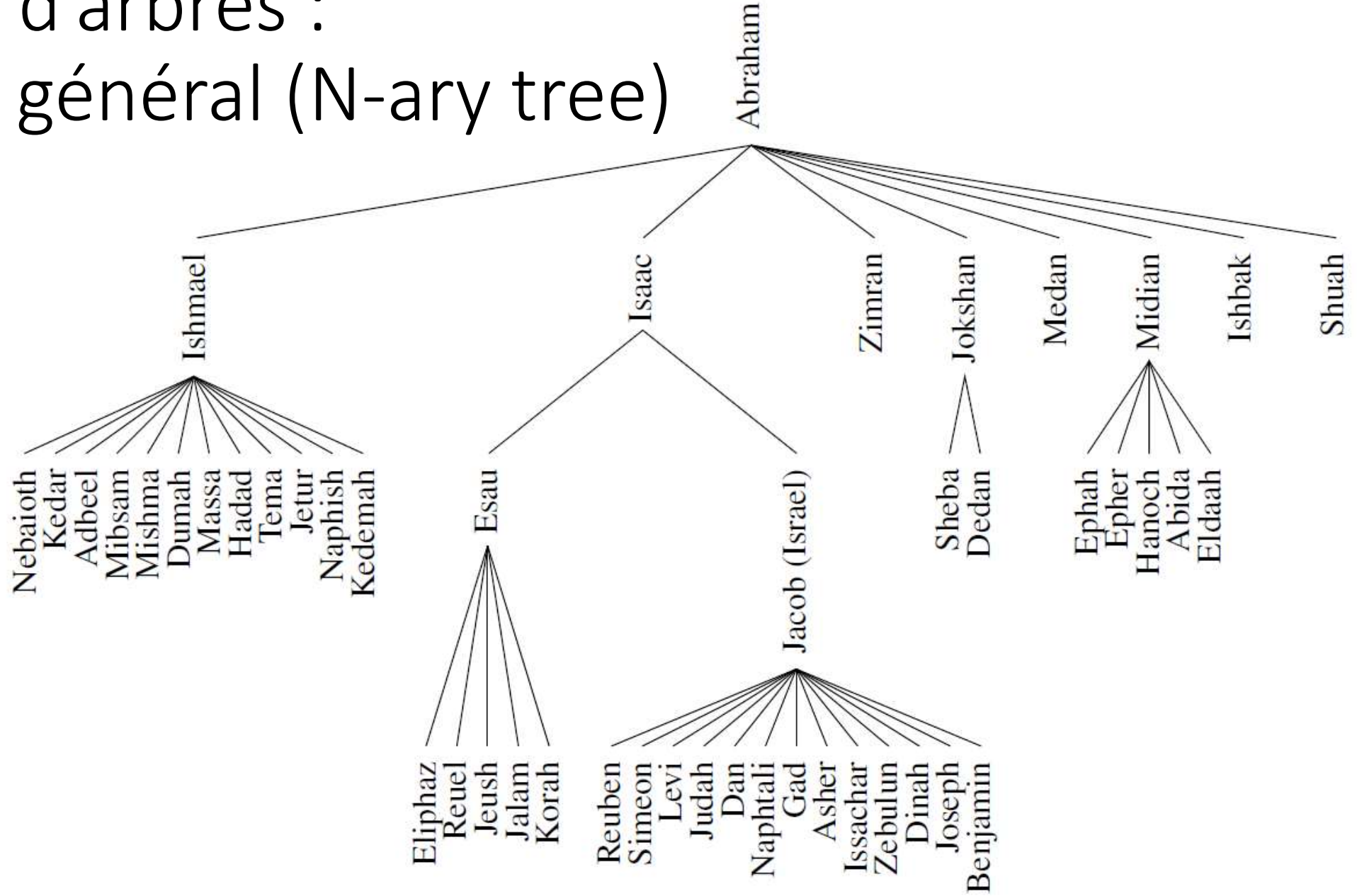
- General tree
- Binary tree
- Binary search tree (BST)
- AVL tree (BST auto-balancé) [1]
- Red-black tree [2]
- Splay tree
- Treap (BST + Heap)
- B-tree [3]

<https://towardsdatascience.com/8-useful-tree-data-structures-worth-knowing-8532c7231e8c>



Types d'arbres :

Arbre général (N-ary tree)



Types d'arbres: Arbre binaire

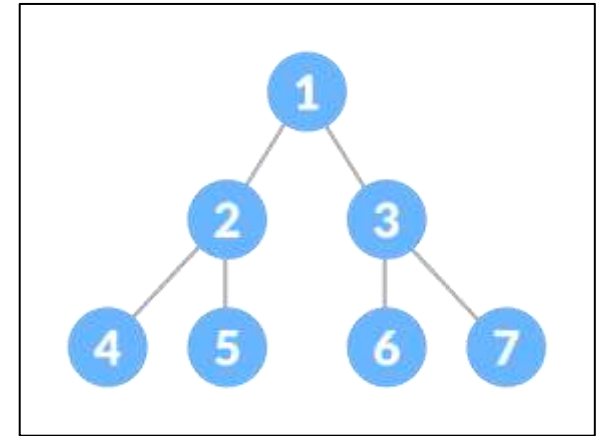
*A **full binary tree** with 16 leaves.
Courtesy of Dr. Shlomit Pinter,
photo taken in Kenya, 2005*



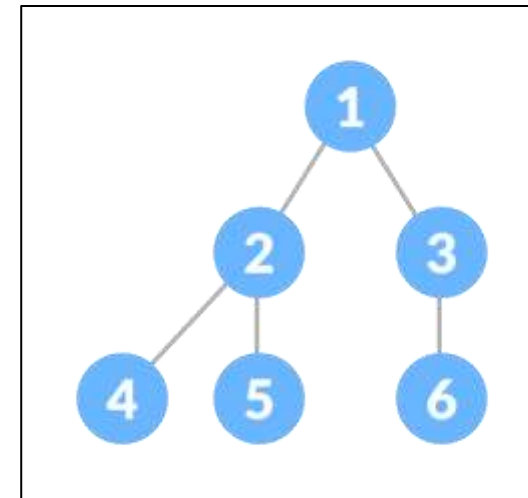
Types d'arbres :

Catégories d'arbres binaires

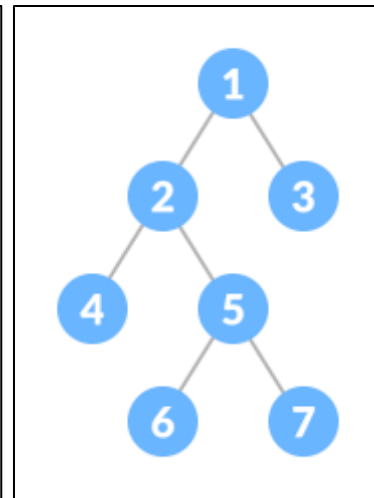
- Chaque nœud possède au plus deux fils.
- Plusieurs catégories d'arbres binaires:
 1. Arbre binaire entier (full Binary Tree): tous les nœuds possèdent absolument deux fils ou rien
 2. Arbre binaire parfait (Perfect Binary Tree) : un arbre binaire où tous les nœuds internes ont exactement deux fils et toutes les feuilles sont de même niveau (profondeur).
 3. Arbre binaire complet (Complete Binary Tree): Chaque niveau est complètement rempli, toutes les feuilles sont rangées à gauche et la dernière feuille peut ne pas avoir un sibling de droite.



Perfect BT



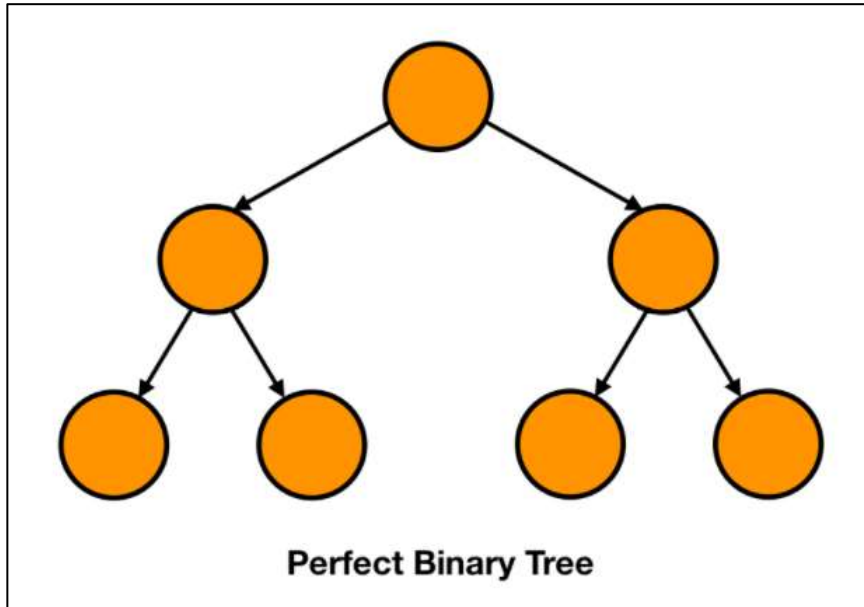
Complete BT



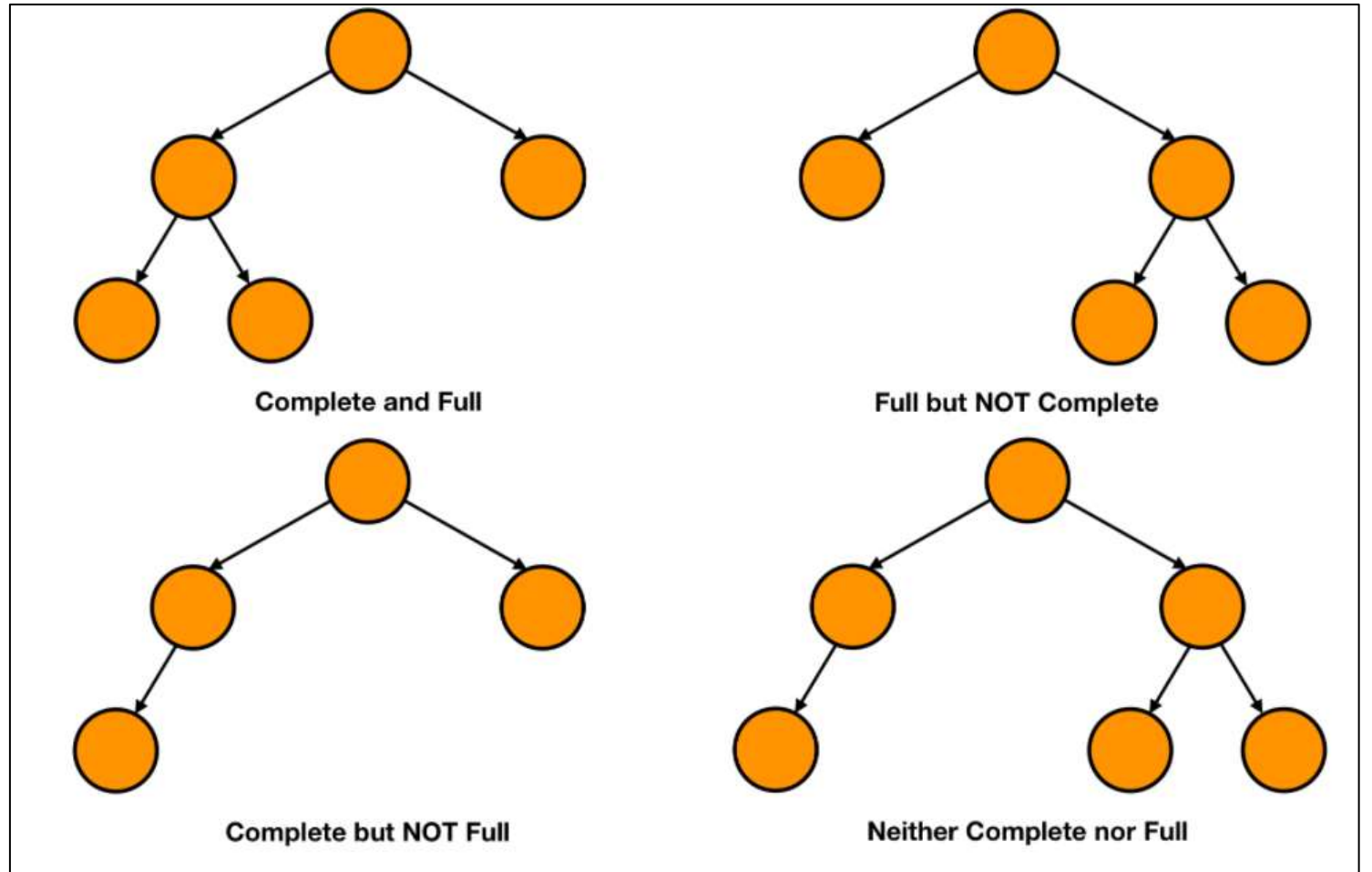
Full BT

Types d'arbres :

Catégories d'arbres binaires (suite)



<https://www.codesdope.com/course/data-structures-binary-trees/>



Tree ADT

p.element(): Return the element stored at position p.

T.root(): Return the position of the root of tree T,
or None if T is empty.

T.is_root(p): Return True if position p is the root of Tree T.

T.parent(p): Return the position of the parent of position p,
or None if p is the root of T.

T.num_children(p): Return the number of children of position p.

T.children(p): Generate an iteration of the children of position p.

T.is_leaf(p): Return True if position p does not have any children.

len(T): Return the number of positions (and hence elements) that
are contained in tree T.

T.is_empty(): Return True if tree T does not contain any positions.

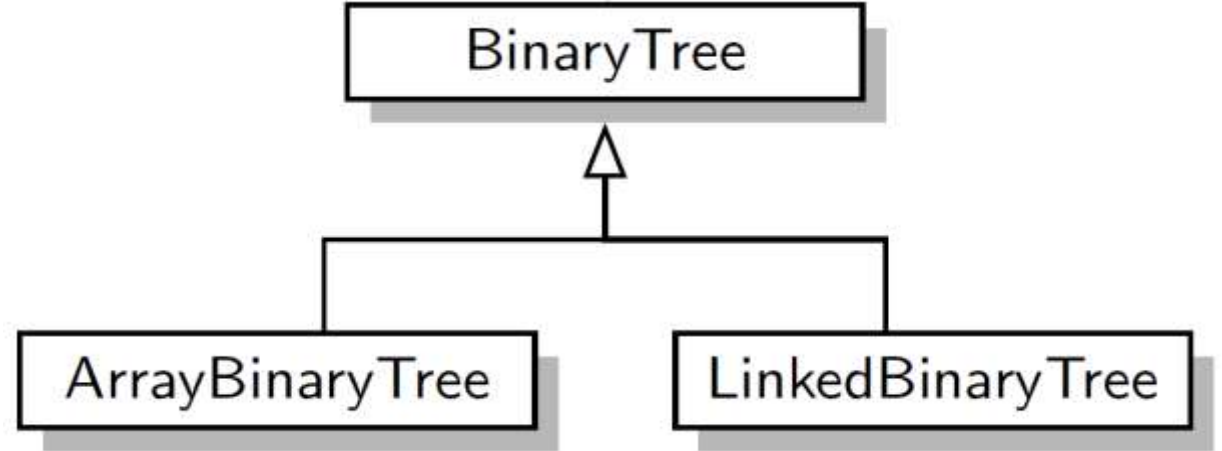
T.positions(): Generate an iteration of all *positions* of tree T.

iter(T): Generate an iteration of all *elements* stored within tree T.

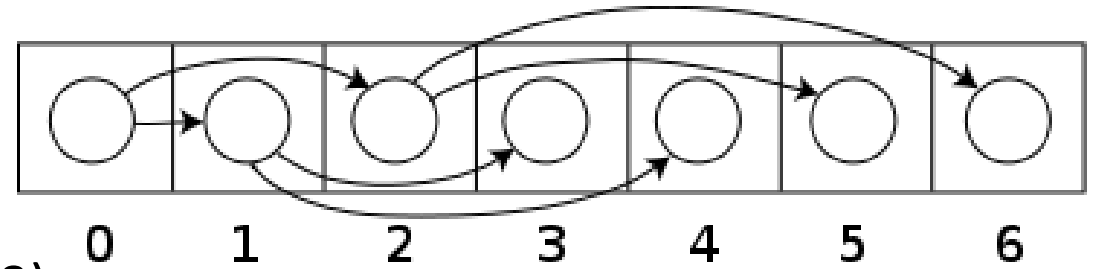
Implémenter un arbre binaire :

Les alternatives

- Deux implémentations:
 - Avec un tableau :
 - Solution simple
 - Accès rapide
 - Perte de mémoire
 - Avec une structure chaînée :
 - Solution un peu plus complexe
 - Opérations moins rapide



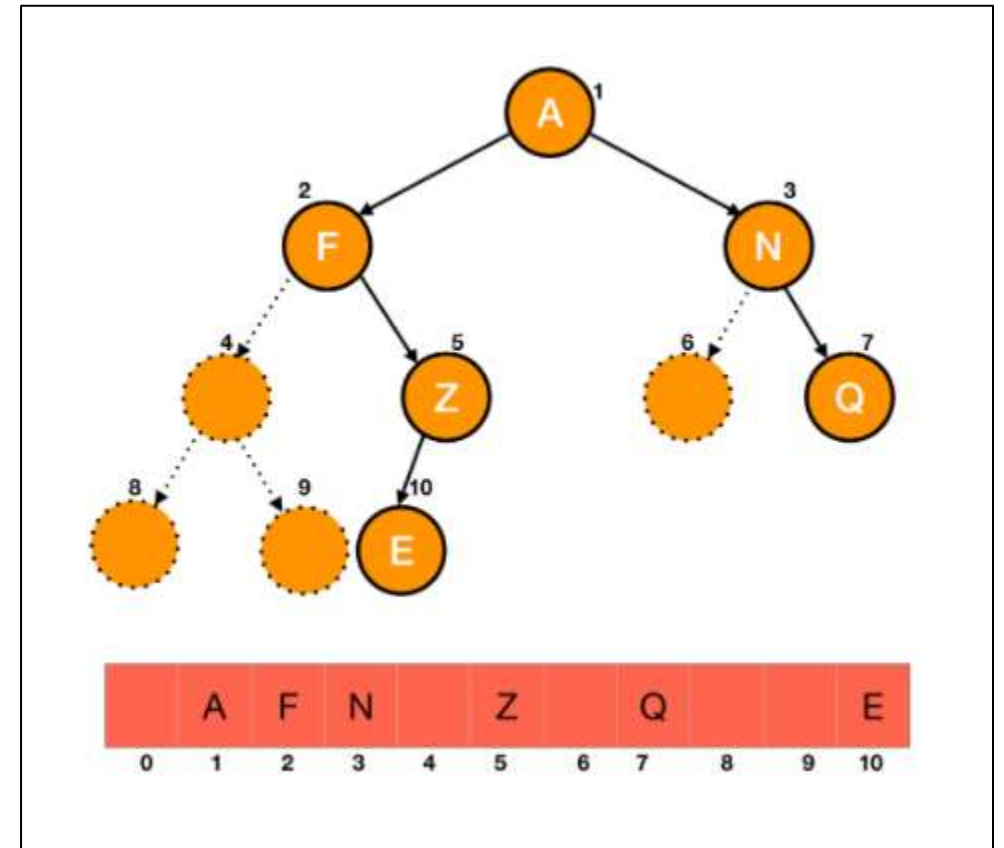
Implémenter un arbre binaire : Structure contiguë



- L'arbre est représenté par un tableau
- La racine est en première position (indice 0)
- Si un nœud est à la position i , alors ses fils se trouvent aux positions $2i+1$ et $2i+2$
- Si l'arbre binaire n'est pas un arbre binaire complet, le tableau contiendra des trous qu'il faudra pouvoir identifier (ex. avec une information spéciale).
- Avantages :
 - Implémentation facile
 - Complexité en temps des opérations : $O(1)$
- Inconvénients :
 - Peut contenir un nombre fixe de nœuds (vue que le tableau est de taille limitée)
 - Si l'arbre a une grande hauteur et peu de nœuds, il peut y avoir un gaspillage de mémoire

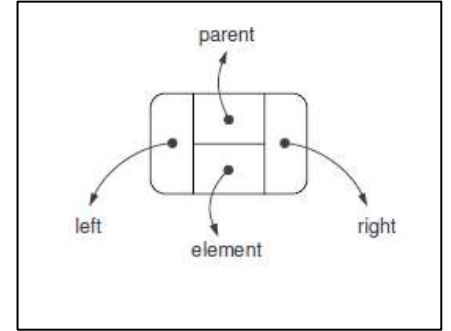
Implémenter un arbre binaire : Structure contiguë – Code Python

```
2 class ArrayBasedBinaryTree(object):
3
4     def __init__(self):
5         self._data = []
6         self._size = 0
7
8     def initialize(self, data, size):
9         self._data = data
10        self._size = size
11
12    def right_child(self, parent_index):
13        if self._data[parent_index] is not None:
14            return (2 * parent_index) + 1
15        return -1
16
17    def left_child(self, parent_index):
18        if self._data[parent_index] is not None:
19            return 2 * parent_index
20        return -1
```

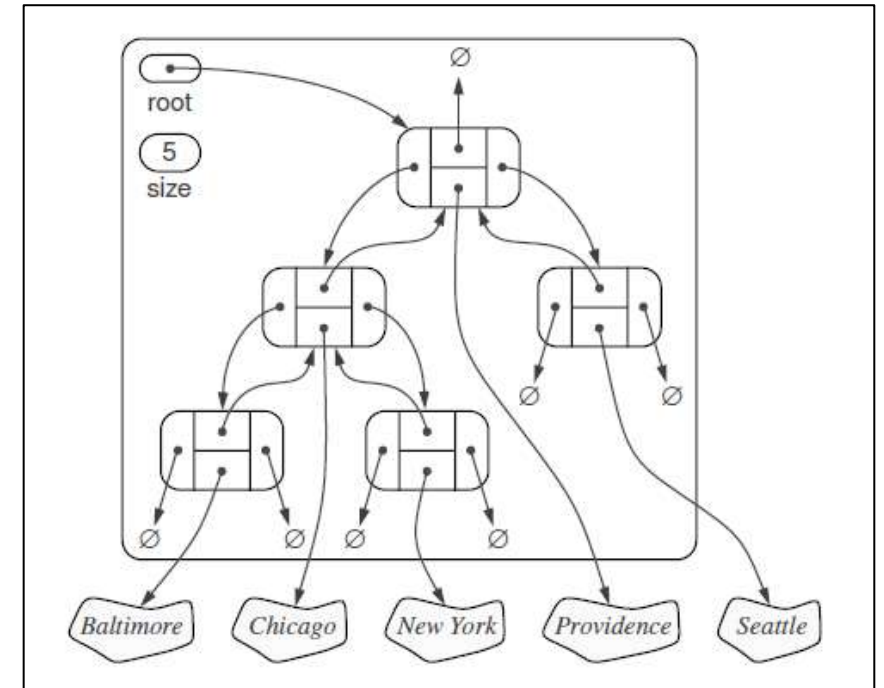


<https://www.codesdope.com/course/data-structures-binary-trees/>

Implémenter un arbre binaire : Structure chaînée



- Structure dynamique
 - Une donnée
 - Des pointeurs vers les fils
 - Un pointeur vers le parent est parfois utile
- Avantages :
 - Peut contenir un nombre variable de nœuds
 - Économie de la mémoire
- Inconvénients :
 - Maîtriser l'allocation dynamique
 - Pas d'accès direct à un nœud de l'arbre (il faut un parcours)



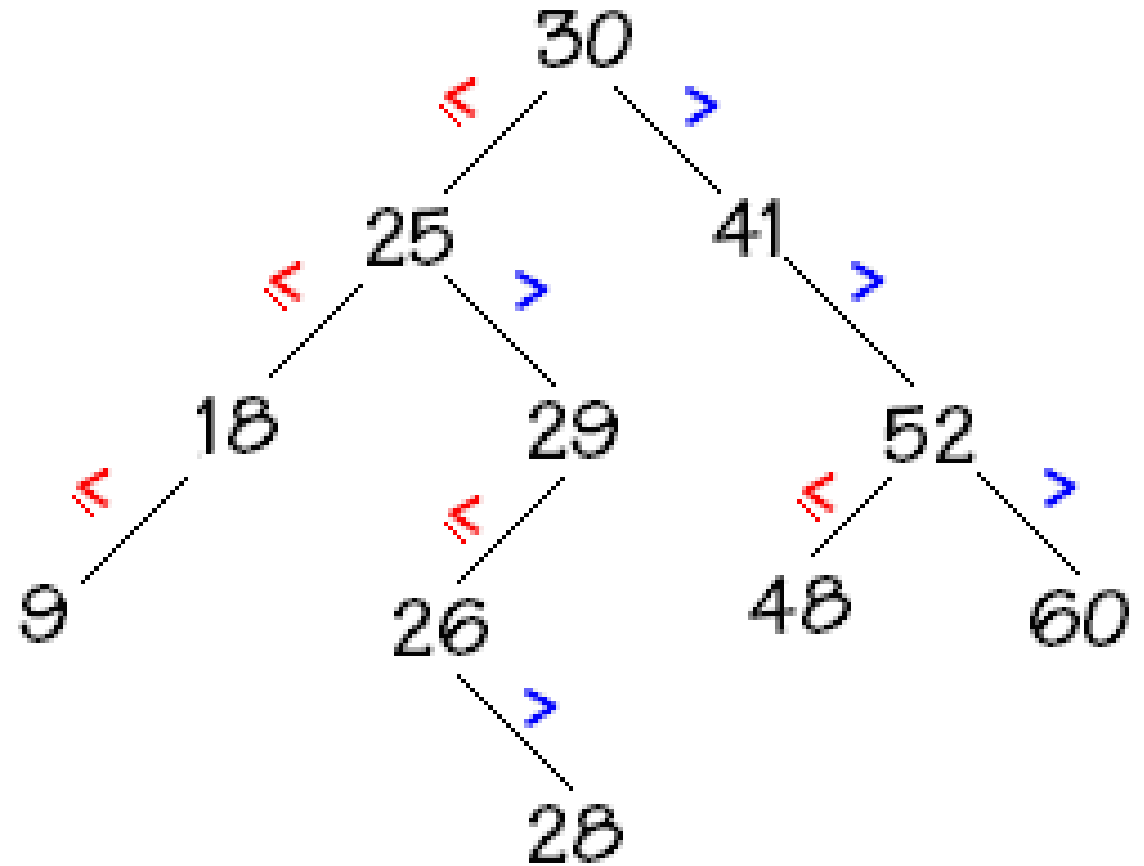
Implémenter un arbre binaire : Structure chaînée – Code Python

```
2 class BinaryNode:
3
4     def __init__(self, value, left=None, right=None):
5         self.data = value
6         self.left = left
7         self.right = right
8
9     def __eq__(self, other):
10         return self.data == other.data
11
12     def __ne__(self, other):
13         return self.data != other.data
14
15     def __gt__(self, other):
16         return self.data > other.data
17
18     def __ge__(self, other):
19         return self.data >= other.data
20
21     def __lt__(self, other):
22         return self.data < other.data
```

```
1 from src.adt.tree.BinaryNode import BinaryNode
2 from src.adt.tree.BinaryTreeHelper import BinaryTreeHelper
3 from src.adt.tree.BinaryTreeWalker import BinaryTreeWalker
4
5
6 class LinkedBinaryTree(object):
7
8     def __init__(self):
9         self._root = None
10        self._size = 0
11
12    def add(self, data):
13        new_node = BinaryNode(data)
14        self._size += 1
15        if self._root is None:
16            self._root = new_node
17            return
18        self.__attach(self._root, new_node)
```

BST

- Un arbre binaire de recherche est un arbre binaire ordonné tel que pour tout nœud :
 - Toutes les valeurs du sous-arbre gauche de ce nœud sont strictement inférieures à la valeur de ce nœud.
 - Toutes les valeurs du sous-arbre droit de ce nœud sont supérieures ou égales à la valeur de ce nœud.

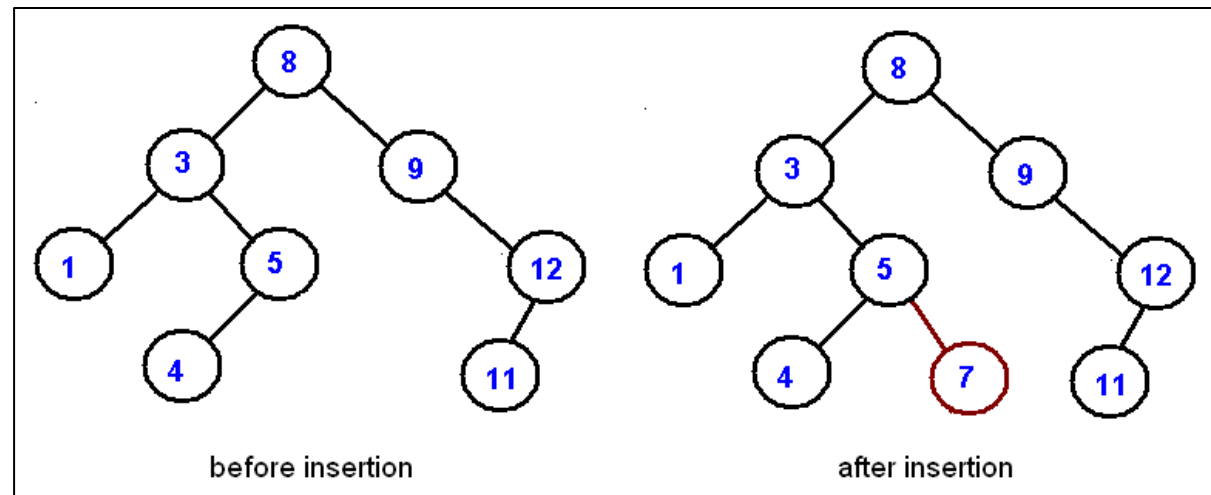


BST :

add(key)

- L'insertion est assez similaire à la recherche. Il faut commencer à la racine et descendre récursivement dans l'arborescence à la recherche d'un emplacement dans le BST pour insérer un nouveau nœud. Si l'élément à insérer est déjà dans l'arborescence, il faut le mettre à jour (remplacer la donnée associée à la clé). Le nouveau nœud remplacera toujours une référence nulle.

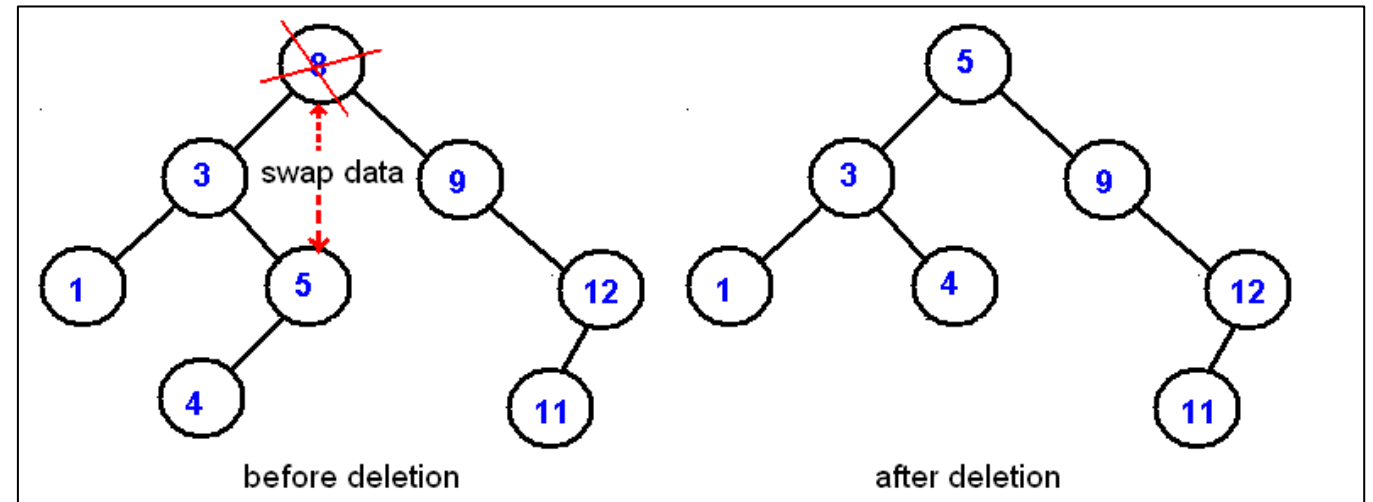
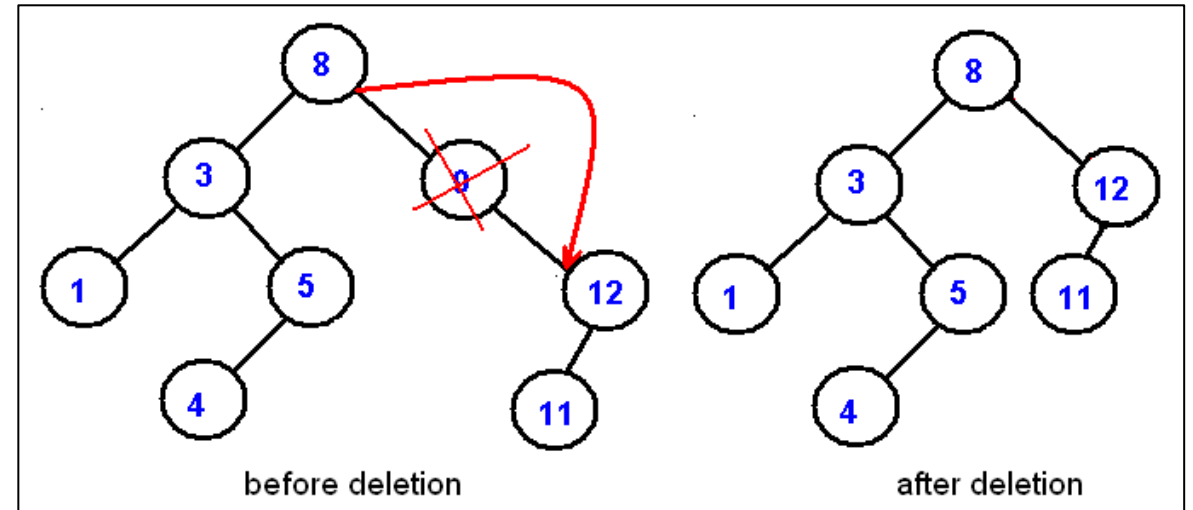
<https://www.cs.cmu.edu/~adamchik/15-121/lectures/Trees/trees.html>



BST:

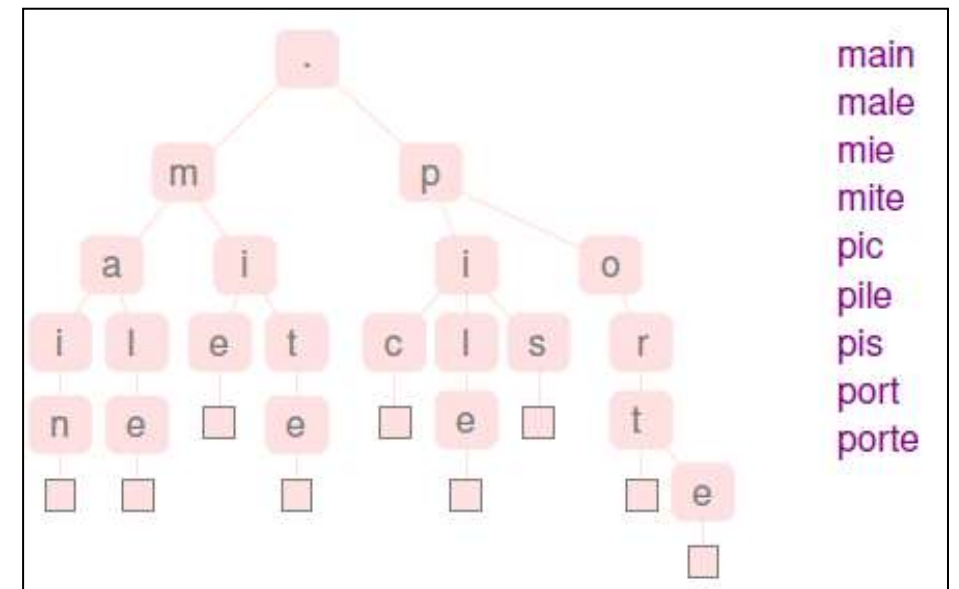
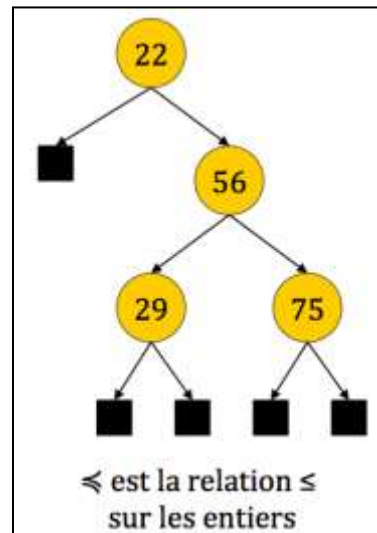
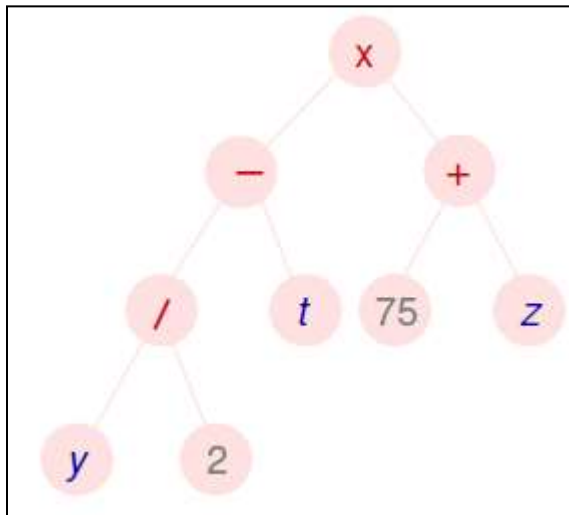
remove(key)

- Plusieurs cas à considérer pour le nœud à supprimer:
 - C'est une feuille : le cas le plus simple
 - A un seul fils : relier l'enfant au parent
 - A deux fils : remplacer le nœud par l'enfant le plus grand dans le sous arbre de gauche (ou le fils le plus petit dans le sous arbre de droite).



Applications de l'arbre:

- Expressions arithmétiques
 - Racine : opération
 - Fils : les opérandes
- Arbre binaire de recherche (BST)
- Arbre syntaxique : pour représenter l'analyse d'une phrase
- Arbre lexicographique : pour représenter un dictionnaire (ensemble de mots).

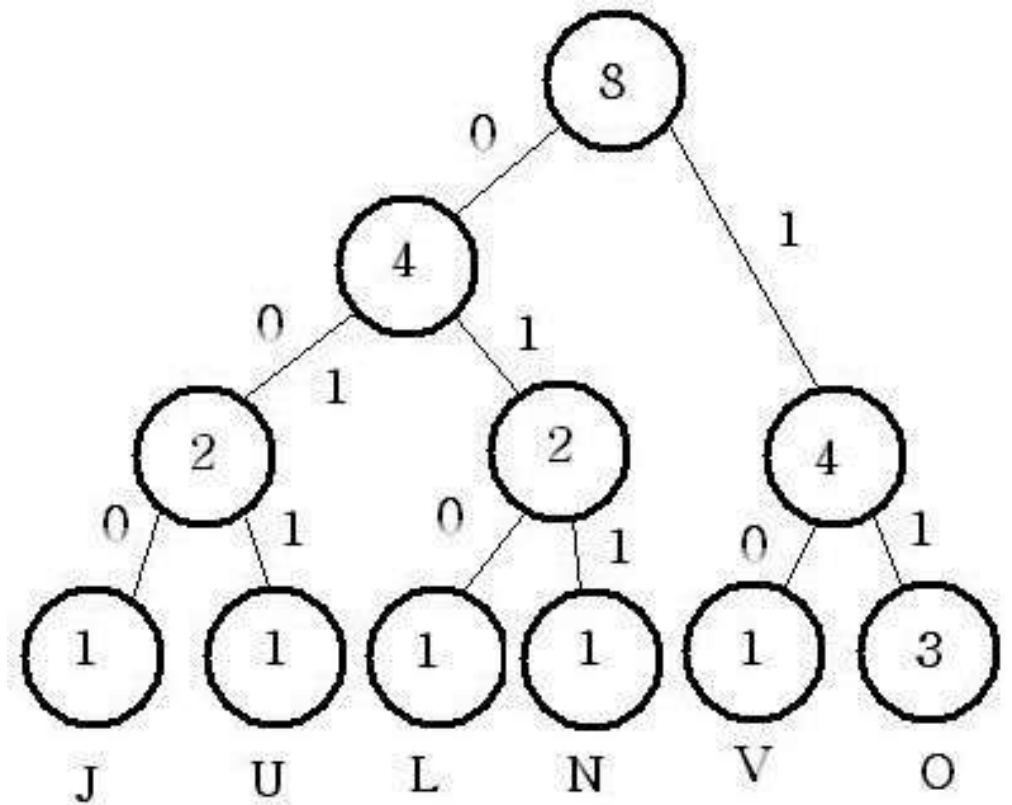


Applications de l'arbre:

- Structure d'un volume de stockage
- Utilisé dans tous les jeux vidéo pour déterminer quels objets devraient être mis en forme (rendered).
- Utilisé avec les algorithmes de compression tels que .jpeg et .mp3
- D'autres applications:
 - <https://stackoverflow.com/questions/2130416/what-are-the-applications-of-binary-trees>

Algorithme de Huffman

- Technique de compression sans perte a hérité du nom de son inventeur, David Huffman.
- Le codage de Huffman est utilisé pour la compression du son au format MP3 et des images aux formats PNG et JPEG.
- Exemple : le mot "JULONOVO" compte 1 fois les lettres "J, U, L, N, V" et 3 fois la lettre "O". Les lettres sont donc codées comme suit : J : 000 - U : 001 - L : 010 - N : 011 - V : 10 - O : 11. Ainsi, Le caractère "O" qui apparaît le plus souvent reçoit un code plus court que les autres.



Complexité avec un arbre binaire

Time Complexity			
	Array	Linked List	BST
Search	$O(n)$	$O(n)$	$O(\log n)$
Insert	$O(1)$	$O(1)$	$O(\log n)$
Remove	$O(n)$	$O(n)$	$O(\log n)$

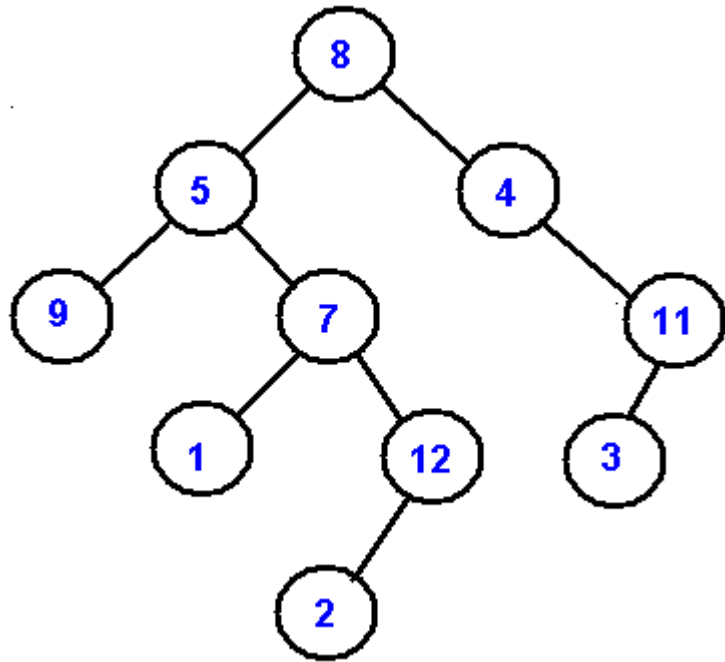
Complément:

Les algorithmes de parcours d'un arbre binaire

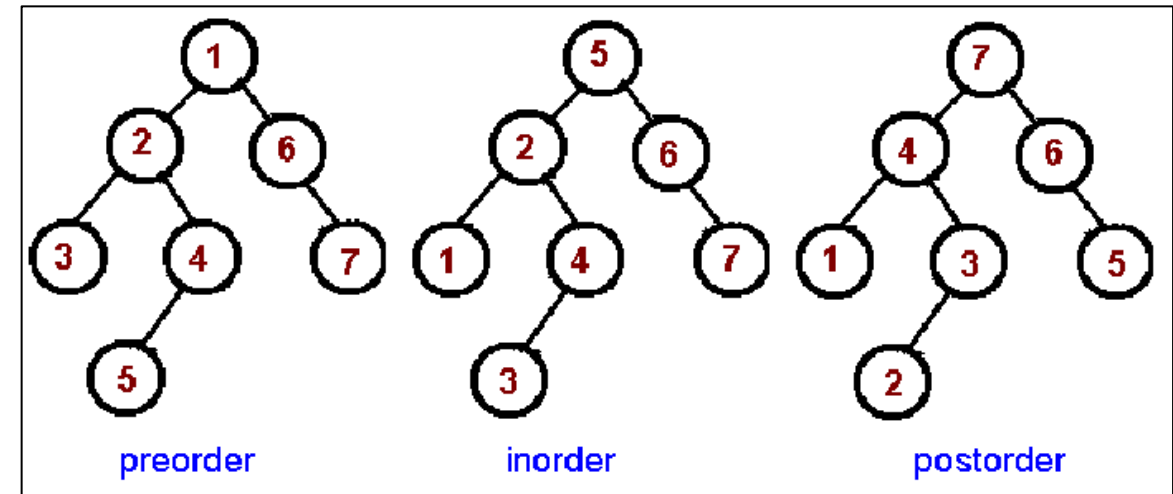
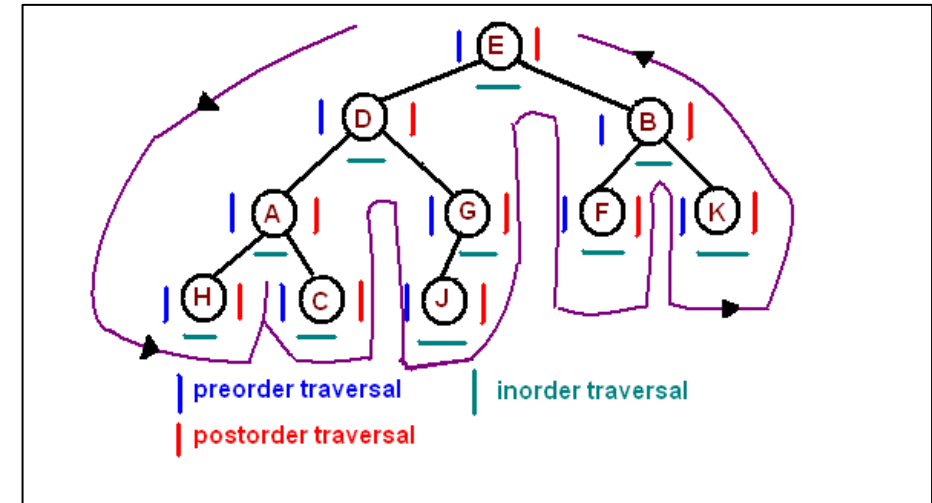
Pre-order, In-order, Post-order

Level-wise (Breadth-first)

Les types de parcours en profondeur d'abord



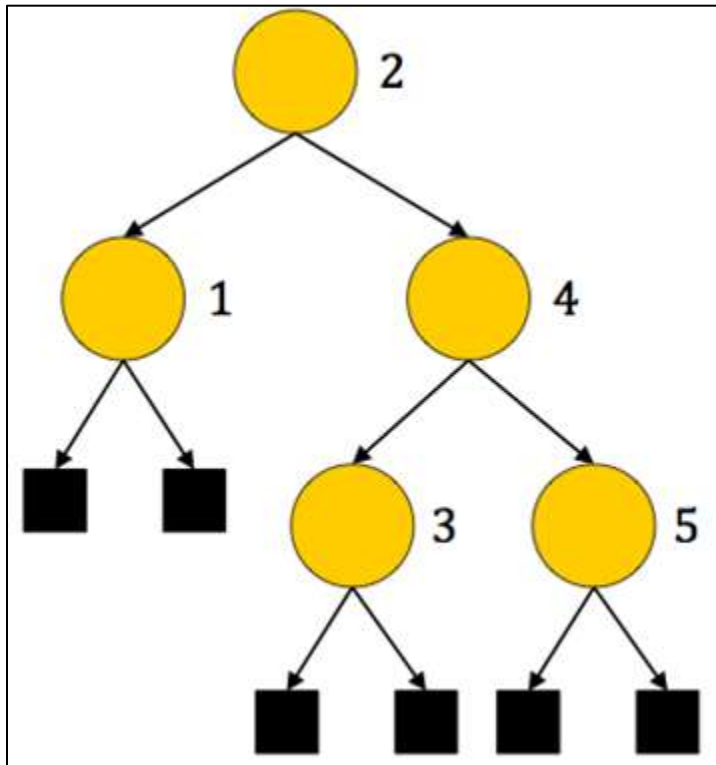
- PreOrder - 8, 5, 9, 7, 1, 12, 2, 4, 11, 3
- InOrder - 9, 5, 1, 7, 2, 12, 8, 4, 3, 11
- PostOrder - 9, 1, 2, 12, 7, 5, 3, 11, 4, 8
- LevelOrder - 8, 5, 4, 9, 7, 11, 1, 12, 3, 2



Parcours d'un arbre binaire :

Parcours en profondeur infixe (in-Order)

- Parcours infixe (en ordre) : Chaque nœud est visité après son fils gauche et avant son fils droit.



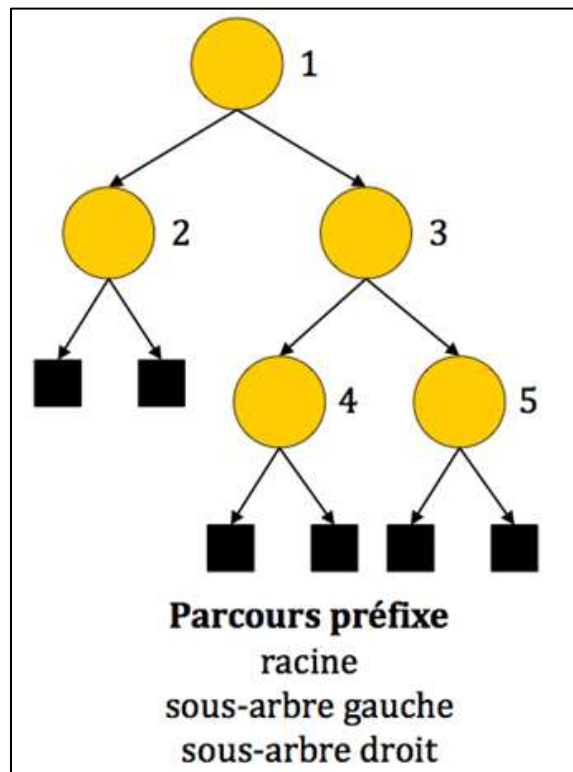
parcours-infixe(a) =
• **si** $\neg$ arbre-vide(a) **alors**
• *parcours-infixe(gauche(a))*
• **traiter** racine(a)
• *parcours-infixe(droit(a))*

INORDER-TREE-WALK(*x*)
1 **if** *x* $\neq$ NIL
2 INORDER-TREE-WALK(*x.left*)
3 print *x.key*
4 INORDER-TREE-WALK(*x.right*)

Parcours d'un arbre binaire :

Parcours en profondeur préfixe (pre-Order)

- Parcours préfixe (en pré-ordre) : chaque nœud est visité avant ses fils.



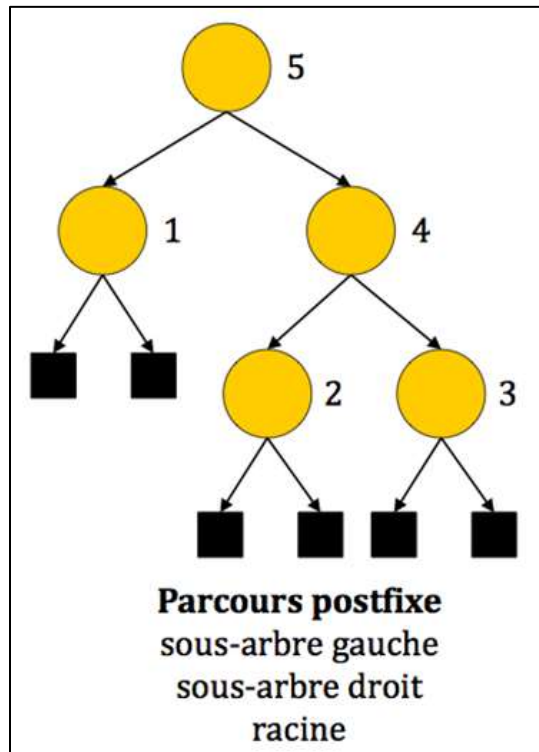
```
parcours-préfixe(a) =  
• si  $\neg$ arbre-vide(a) alors  
  • traiter racine(a)  
  • parcours-préfixe(gauche(a))  
  • parcours-préfixe(droit(a))
```

```
PREORDER-TREE-WALK(x)  
1  if  $x \neq \text{NIL}$   
2    print x.key  
3    PREORDER-TREE-WALK(x.left)  
4    PREORDER-TREE-WALK(x.right)
```

Parcours d'un arbre binaire :

Parcours en profondeur postfixe (Post-Order)

- Parcours postfixe (en post-ordre) : chaque nœud est visité après ses deux fils.

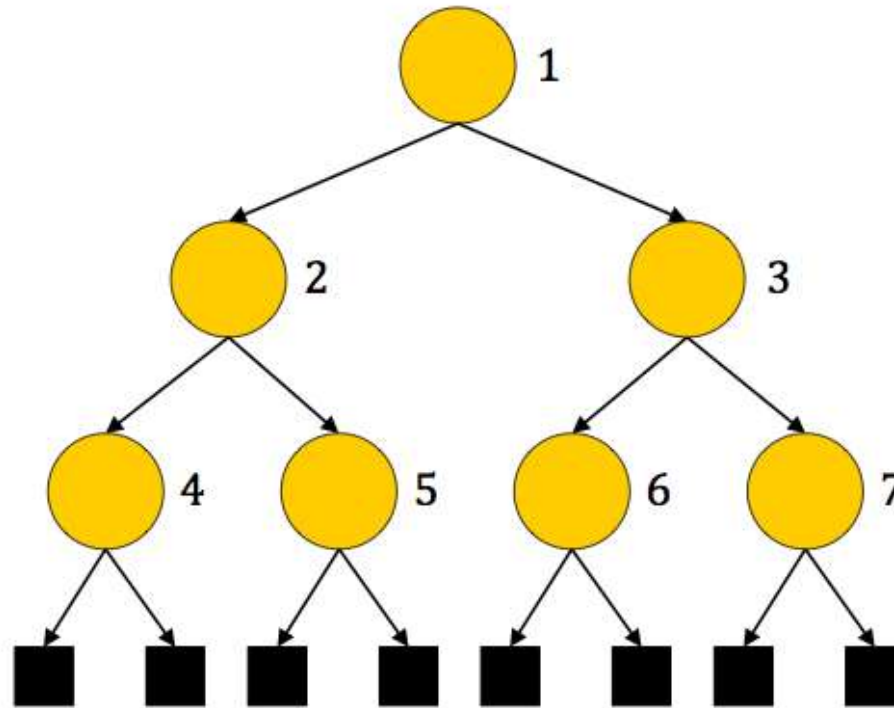


```
parcours-postfixe(a) =  
• si  $\neg$ arbre-vide(a) alors  
  • parcours-postfixe(gauche(a))  
  • parcours-postfixe(droit(a))  
  • traiter racine(a)
```

```
POSTORDER-TREE-WALK(x)  
1  if x  $\neq$  NIL  
2    POSTORDER-TREE-WALK(x.left)  
3    POSTORDER-TREE-WALK(x.right)  
4    print x.key
```

Parcours d'un arbre binaire :

Parcours en largeur (Breadth-First Traversal)



Parcours en largeur
niveau par niveau

parcours-en-largeur(a) =

- **si** $\neg \text{arbre-vide}(a)$ **alors**

- *f* = file-vide

- *entrer*(*f*, *a*)

- **faire**

- *a* = *sortir*(*f*)

- **traiter** *racine*(*a*)

- *g* = *gauche*(*a*)

- *d* = *droit*(*a*)

- **si** $\neg \text{arbre-vide}(g)$ **alors**

- *entrer*(*f*, *g*)

- **si** $\neg \text{arbre-vide}(d)$ **alors**

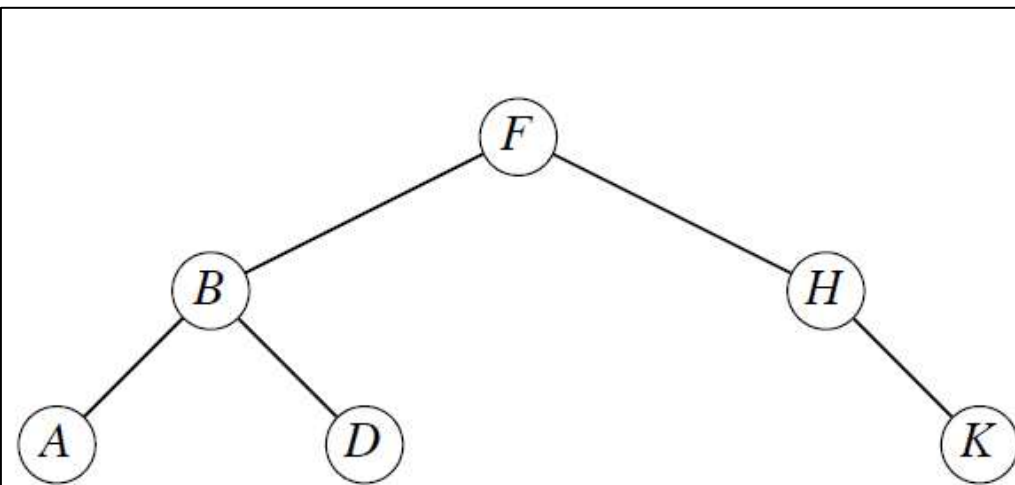
- *entrer*(*f*, *d*)

- **jusqu'à** *file-vide*(*f*)

Parcours d'un arbre binaire :

Parcours en largeur (Breadth-First Traversal)

- Implémentation à l'aide d'une file en $O(n)$



$\Rightarrow \langle F, B, H, A, D, K \rangle$

BREADTH-TREE-WALK(x)

```
1  Q = "Empty queue"
2  ENQUEUE(Q, x)
3  while not QUEUE-EMPTY(Q)
4      y = DEQUEUE(Q)
5      print y.key
6      if y.left  $\neq$  NIL
7          ENQUEUE(Q, y.left)
8      if y.right  $\neq$  NIL
9          ENQUEUE(Q, y.right)
```

Références

- <https://www.baeldung.com/java-binary-tree>
- https://www.youtube.com/watch?v=j-TSLUFV_CY
- <https://opendatastructures.org/versions/edition-0.1d/ods-java/node36.html>
- <https://medium.com/quick-code/data-structures-traversing-trees-9473f6d9f4ef>
- <https://www.programiz.com/dsa/trees>
- <https://runestone.academy/runestone/books/published/pythonds/Trees/toctree.html>
- <https://www.codesdope.com/course/data-structures-binary-trees/>